Complete System Design Interview Questions & Answers Guide

Table of Contents

Beginner Level Questions

1. What is System Design?

Answer Format:

> Definition → Purpose → Key Goals → Simple Example

Sample Answer: "System design is the process of creating the architecture for large-scale software systems. It's like creating a blueprint for a building before construction.

Purpose: To build systems that can handle millions of users without breaking.

Key Goals:

- **Scalability:** Handle more users as the app grows
- **Reliability:** System works 99.9% of the time
- **Performance:** Fast response times (under 2 seconds)
- **Maintainability:** Easy to update and fix

Simple Example: Think of WhatsApp - it needs to handle 2 billion users sending messages simultaneously. System design helps plan how to build servers, databases, and connections to make this possible."

---

2. What is the difference between Vertical and Horizontal Scaling?

Answer Format:

> Definition of Both → Real Example → Pros/Cons → When to Use

Sample Answer: "**Vertical Scaling (Scale Up):** Adding more power to one machine **Horizontal Scaling (Scale Out):** Adding more machines

Real Example:

- **Vertical:** Your laptop is slow, so you add more RAM (4GB → 16GB)
- **Horizontal:** Your website is slow, so you add more servers (1 server → 5 servers)

Comparison:

Vertical Scaling	Horizontal Scaling
Easier to implement	More complex setup
Hardware limits	No limits
Single point of failure	Fault tolerant
Example: Upgrade CPU	Example: Add more servers

When to Use:

- **Vertical:** Small applications, quick fixes
- **Horizontal:** Large applications, long-term growth"

---

3. What is Load Balancing and why do we need it?

Answer Format:

> Definition → Problem it Solves → How it Works → Benefits

Sample Answer: "**Load Balancing** distributes incoming requests across multiple servers so no single server gets overwhelmed.

Problem it Solves: Imagine a restaurant with 1 waiter serving 100 customers - chaos! Load balancing is like having 5 waiters, each serving 20 customers.

How it Works:

1. User sends request to load balancer
2. Load balancer picks the best server
3. Server processes request
4. Response sent back to user

Real Example:

```
Before: [1000 Users] → [1 Server] → Server crashes
After:  [1000 Users] → [Load Balancer] → [Server 1: 200 users]
                                       → [Server 2: 200 users]
                                       → [Server 3: 200 users]
                                       → [Server 4: 200 users]
                                       → [Server 5: 200 users]
```

Benefits:

- No single point of failure
- Better performance
- Easy to add more servers"

---

4. What is Caching? Give examples.

Answer Format:

```
Definition → Why Needed → Types → Real Examples → Benefits
```

Sample Answer: "**Caching** stores frequently used data in fast storage so you don't have to fetch it from slow storage every time.

Real-Life Analogy: You keep important phone numbers in your phone's contacts (cache) instead of looking them up in a phone book (database) every time.

Why Needed:

- Database queries are slow (2-3 seconds)
- Users expect instant responses
- Reduces server load

Types with Examples:

1. **Browser Cache:** Your browser saves images from websites
2. **CDN Cache:** Netflix stores popular movies closer to users
3. **Database Cache:** Instagram caches your feed for quick loading
4. **Application Cache:** Spotify caches recently played songs

Benefits:

- **Speed:** 10x faster response times
- **Cost:** Less database usage = lower costs
- **User Experience:** Instant loading

When to Use:

- Data that doesn't change often (user profiles)
- Expensive database queries
- Popular content (trending posts)"

---

5. SQL vs NoSQL - When to use which?

Answer Format:

> Definition of Both → Key Differences → When to Use → Examples

Sample Answer: "**SQL (Structured Query Language):** Data stored in tables with rows and columns **NoSQL (Not Only SQL):** Data stored in flexible formats

Key Differences:

SQL	NoSQL
Fixed structure	Flexible structure
ACID properties	Eventually consistent
Complex queries	Simple queries
Vertical scaling	Horizontal scaling

When to Use SQL:

- **Banking systems:** Need exact money calculations
- **E-commerce:** Orders, customers, products have relationships
- **CRM systems:** Structured business data
- **Example:** PostgreSQL, MySQL

When to Use NoSQL:

- **Social media:** Posts can have text, images, videos
- **IoT applications:** Millions of sensor readings

- **Real-time analytics:** Need fast reads/writes
- **Example:** MongoDB, Cassandra

Real Decision Example:

```
Scenario: Building a blog platform
- User profiles: SQL (structured data)
- Blog posts: NoSQL (flexible content)
- Comments: SQL (relationships between users and posts)
```"

---

6. What is a Microservice? How is it different from Monolithic?

Answer Format:
```
Definition → Comparison → Pros/Cons → When to Use
```

Sample Answer: "**Monolithic:** One big application with all features together **Microservices:** Breaking the application into small, independent services

Restaurant Analogy:

- **Monolithic:** One chef does everything (cooking, serving, billing)
- **Microservices:** Separate chef, waiter, cashier

Comparison:

Monolithic	Microservices
One codebase	Multiple codebases
Single deployment	Independent deployments
Shared database	Separate databases
One technology	Multiple technologies

Pros/Cons: Monolithic Pros: Simple to start, easy testing **Monolithic Cons:** Hard to scale, one failure affects all

Microservices Pros: Independent scaling, fault isolation **Microservices Cons:** Complex networking, data consistency

When to Use Microservices:

- Large teams (>10 people)
- Different parts need different technologies
- Need to scale parts independently
- Example: Netflix has 500+ microservices

Real Example - E-commerce:

```
Monolithic: One app handles users, products, orders, payments
Microservices:
- User Service (handles login)
- Product Service (manages catalog)
- Order Service (processes orders)
- Payment Service (handles payments)
```"

Intermediate Level Questions

7. Explain different types of Load Balancing algorithms.

Answer Format:
```
List Types → Explain Each → Use Cases → Examples
```

Sample Answer: "Load balancing algorithms determine how requests are distributed across servers.

1. Round Robin

- **How it works:** Requests sent to servers in circular order
- **Example:** Request 1→Server A, Request 2→Server B, Request 3→Server C, Request 4→Server A...
- **Use case:** All servers have similar capacity
- **Pros:** Simple, fair distribution
- **Cons:** Doesn't consider server load

2. Weighted Round Robin

- **How it works:** Servers assigned weights based on capacity
- **Example:** Server A (weight 3), Server B (weight 1) → A gets 3 requests for every 1 to B
- **Use case:** Servers with different capacities
- **Real scenario:** New powerful server + old weaker server

3. Least Connections

- **How it works:** Route to server with fewest active connections
- **Example:** Server A (5 connections), Server B (2 connections) → Next request goes to B
- **Use case:** Long-running requests (video streaming, file downloads)
- **Benefit:** Handles varying request duration

4. IP Hash

- **How it works:** Hash client IP to determine server
- **Example:** IP 192.168.1.100 always goes to Server A
- **Use case:** Session sticky applications (shopping carts)
- **Benefit:** Same user always hits same server

5. Geographic

- **How it works:** Route based on user location
- **Example:** US users → US servers, EU users → EU servers
- **Use case:** Global applications
- **Benefit:** Reduced latency

Choosing Algorithm:

- **Static websites:** Round Robin
- **Shopping sites:** IP Hash (for sessions)
- **Streaming services:** Least Connections
- **Global apps:** Geographic"

---

8. What is Database Sharding? Explain different sharding strategies.

Answer Format:

> Definition → Why Needed → Strategies → Challenges → Examples

Sample Answer: "**Database Sharding** splits large database into smaller pieces (shards) across multiple servers.

Why Needed:

- Single database can't handle millions of records
- Queries become slow (>5 seconds)

- Database server reaches hardware limits

Library Analogy: Instead of one giant library, create multiple smaller libraries in different areas of the city.

Sharding Strategies:

1. Horizontal Sharding (Most Common)

- **Split:** Rows across databases
- **Example:**
  - Shard 1: Users with ID 1-1000
  - Shard 2: Users with ID 1001-2000
  - Shard 3: Users with ID 2001-3000
- **Use case:** User data, order history

2. Vertical Sharding

- **Split:** Columns across databases
- **Example:**
  - Shard 1: User basic info (name, email)
  - Shard 2: User preferences (settings, themes)
  - Shard 3: User activity (login history, actions)
- **Use case:** Wide tables with many columns

3. Functional Sharding

- **Split:** By feature/service
- **Example:**
  - Shard 1: User management
  - Shard 2: Product catalog
  - Shard 3: Order processing
- **Use case:** Microservices architecture

4. Geographic Sharding

- **Split:** By location
- **Example:**
  - Shard 1: US users
  - Shard 2: EU users
  - Shard 3: Asia users

- **Use case:** Global applications

Challenges:

1. **Cross-shard queries:** Joining data from multiple shards
2. **Rebalancing:** Moving data when adding/removing shards
3. **Hotspots:** Uneven data distribution
4. **Complexity:** Application logic becomes complex

Real Example - Instagram:

```
Sharding Strategy: By user ID
- Shard 1: User IDs 1-10M
- Shard 2: User IDs 10M-20M
- Shard 3: User IDs 20M-30M

Query: "Get user with ID 15M" → Route to Shard 2
```"

---

9. Explain Message Queues. When and how to use them?

Answer Format:
```
Definition → Problem it Solves → How it Works → Types → Examples
```

Sample Answer: "**Message Queues** enable asynchronous communication between services by storing messages temporarily.

Post Office Analogy:

- You write a letter (message) and put it in mailbox (queue)
- Postal service (queue system) delivers when convenient
- Recipient gets letter when they check mailbox

Problem it Solves: Without queues:

```
User clicks "Send Email" → App sends email → User waits 5 seconds → Success
```

With queues:

```
User clicks "Send Email" → App queues email → User sees "Success" immediately
Background: Queue processes email separately
```

How it Works:

1. **Producer** creates message and sends to queue
2. **Queue** stores message temporarily
3. **Consumer** picks up message and processes
4. **Acknowledgment** sent back to queue when done

Types:

1. Point-to-Point

- One producer, one consumer
- Message consumed once
- **Example:** Order processing

```
Order Service → [Order Queue] → Payment Service
```

2. Publish-Subscribe

- One producer, multiple consumers
- Message copied to all subscribers
- **Example:** User registration

```
User Service → [Event Queue] → Email Service
                             → Analytics Service
                             → Notification Service
```

Popular Queue Systems:

- **RabbitMQ:** Feature-rich, reliable
- **Apache Kafka:** High-throughput, distributed
- **Redis:** Fast, simple
- **AWS SQS:** Managed, scalable

When to Use:

1. **Long-running tasks:** Image processing, email sending
2. **Decoupling services:** Services don't need to know about each other

3. **Load leveling:** Handle traffic spikes
4. **Reliability:** Don't lose data if service goes down

Real Examples:

- **Uber:** Queue ride requests during peak hours
- **Instagram:** Queue photo processing after upload
- **Netflix:** Queue video encoding for new uploads"

---

10. What is CAP Theorem? Explain with examples.

Answer Format:

> Definition → Three Properties → Trade-offs → Real Examples

Sample Answer: "**CAP Theorem** states that in a distributed system, you can only guarantee 2 out of 3 properties:

C - Consistency: All nodes see the same data at the same time **A - Availability:** System remains operational **P - Partition Tolerance:** System continues despite network failures

Real-World Analogy: Imagine a bank with 3 branches. CAP theorem says you can't have all three:

- All branches show same balance (Consistency)
- All branches always open (Availability)
- Branches work even if phones are down (Partition Tolerance)

The Trade-offs:

1. CP Systems (Consistency + Partition Tolerance)

- **Sacrifice:** Availability
- **Example:** Banking systems
- **Behavior:** If network fails, some branches close rather than show wrong balance
- **Real system:** MongoDB (in certain configurations)

2. AP Systems (Availability + Partition Tolerance)

- **Sacrifice:** Consistency
- **Example:** Social media feeds
- **Behavior:** All servers stay up, but may show slightly different data
- **Real system:** Cassandra, DynamoDB

3. CA Systems (Consistency + Availability)

- **Sacrifice:** Partition Tolerance
- **Example:** Traditional single-machine databases
- **Limitation:** Not truly distributed
- **Real system:** MySQL, PostgreSQL (single instance)

Real Examples:

Facebook (AP System):

```
Scenario: You post a photo
- US servers: Show your photo immediately
- EU servers: May take 5 minutes to show your photo
- Trade-off: Availability over consistency
```

Bank Transfer (CP System):

```
Scenario: Transfer $100 between accounts
- If network fails: Transaction blocked
- Ensures: No money lost or duplicated
- Trade-off: Consistency over availability
```

Choosing Your Trade-off:

- **Financial systems:** Choose CP (accuracy over availability)
- **Social media:** Choose AP (user experience over perfect consistency)
- **E-commerce:** Mix of both (inventory needs consistency, recommendations can be eventual)"

---

11. What is Database Replication? Explain Master-Slave vs Master-Master.

Answer Format:

```
Definition → Why Needed → Types → Comparison → Use Cases
```

Sample Answer: "**Database Replication** creates copies of data across multiple database servers.

Why Needed:

- **Backup:** If main database fails, replica takes over
- **Performance:** Spread read load across multiple servers

- **Geography:** Serve users from closest database location

Types:

1. Master-Slave Replication

How it Works:

- **Master:** Handles all writes (INSERT, UPDATE, DELETE)
- **Slaves:** Handle only reads (SELECT)
- **Replication:** Master sends changes to slaves

```
Application → [Master DB] → [Slave 1]
                  → [Slave 2]
                  → [Slave 3]

Writes: Go to Master
Reads: Go to any Slave
```

Pros:

- Simple to implement
- Read scalability
- Data backup

Cons:

- Single point of failure for writes
- Replication lag (slaves slightly behind)
- Master can become bottleneck

2. Master-Master Replication

How it Works:

- **Multiple Masters:** All can handle reads and writes
- **Bi-directional:** Masters sync with each other

```
Application → [Master 1] ←→ [Master 2]
                  ←→ [Master 3]

Writes: Can go to any Master
Reads: Can go to any Master
```

Pros:

- No single point of failure
- Write scalability
- High availability

Cons:

- Complex conflict resolution
- Potential data inconsistency
- More complex to maintain

Real-World Examples:

Netflix (Master-Slave):

```
Use Case: Movie recommendations
- Master: Stores user ratings, preferences
- Slaves: Serve movie recommendations to users
- Why: Many reads, fewer writes
```

WhatsApp (Master-Master):

```
Use Case: Message delivery
- Master US: Handles US user messages
- Master EU: Handles EU user messages
- Sync: Messages sync between regions
- Why: Need high availability globally
```

When to Choose:

Choose Master-Slave when:

- Read-heavy applications (blogs, news sites)
- Simple consistency requirements
- Small-medium scale

Choose Master-Master when:

- Global applications
- High availability requirements
- Complex write patterns

Common Issues:

- **Replication Lag:** Slaves behind master by seconds
- **Conflict Resolution:** What if same data updated in two masters?
- **Failover:** How to switch when master fails?"

Advanced Level Questions

12. How would you design a URL shortener like bit.ly?

Answer Format:

```
Requirements → High-Level Design → Database Design → Scaling → Deep Dive
```

Sample Answer: "I'll design a URL shortener like bit.ly step by step.

1. Requirements Gathering

Functional Requirements:

- Shorten long URLs to 7-character codes
- Redirect short URLs to original URLs
- Custom aliases (optional)
- Analytics (click tracking)

Non-Functional Requirements:

- **Scale:** 100M URLs created daily
- **Availability:** 99.9% uptime
- **Performance:** Redirection <100ms
- **Storage:** 5 years retention

2. Capacity Estimation

```
Daily URL creation: 100M
Read/Write ratio: 100:1 (more clicks than creation)
Daily redirections: 10B
Storage per URL: 500 bytes
Total storage: 100M × 365 × 5 × 500 bytes = 91TB
```

3. High-Level Design

```
[User] → [Load Balancer] → [Web Servers] → [Cache] → [Database]
                                  → [Analytics Service]
                                  → [CDN (for static content)]
```

4. Database Design

URL Table:

```sql
CREATE TABLE urls (
    id BIGINT PRIMARY KEY,
    short_url VARCHAR(7) UNIQUE,
    long_url TEXT,
    user_id BIGINT,
    created_at TIMESTAMP,
    expires_at TIMESTAMP,
    is_custom BOOLEAN
);

CREATE INDEX idx_short_url ON urls(short_url);
```

Analytics Table:

```sql
CREATE TABLE clicks (
    id BIGINT PRIMARY KEY,
    short_url VARCHAR(7),
    clicked_at TIMESTAMP,
    ip_address VARCHAR(45),
    user_agent TEXT,
    country VARCHAR(2)
);
```

5. URL Encoding Algorithm

Approach 1: Base62 Encoding

```python
```

```
def encode_url(id):
    characters = "0123456789abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ"
    base = len(characters)

    if id == 0:
        return characters[0]

    result = ""
    while id > 0:
        result = characters[id % base] + result
        id = id // base

    return result.ljust(7, '0')  # Pad to 7 characters
```

Approach 2: Random Generation

```
python

import random
import string

def generate_short_url():
    return ''.join(random.choices(string.ascii_letters + string.digits, k=7))
```

6. API Design

Create Short URL:

```
POST /api/v1/shorten
Request: {"long_url": "https://example.com/very/long/url"}
Response: {"short_url": "abc123d", "long_url": "https://example.com/very/long/url"}
```

Redirect:

```
GET /abc123d
Response: 301 Redirect to original URL
```

7. Caching Strategy

Cache Popular URLs:

```
Cache Layer: Redis
Key: short_url
Value: long_url
TTL: 24 hours
Cache Size: 20% of total URLs (80/20 rule)
```

8. Scaling Solutions

Database Sharding:

```
Shard by: First character of short_url
Shard 1: URLs starting with 0-9, a-f
Shard 2: URLs starting with g-m
Shard 3: URLs starting with n-s
Shard 4: URLs starting with t-z, A-Z
```

Read Replicas:

```
Master: Handle writes (URL creation)
Slaves: Handle reads (URL redirections)
Ratio: 1 Master : 5 Slaves
```

9. Analytics Service

Real-time Analytics:

```
Message Queue: Kafka
Producer: Web servers send click events
Consumer: Analytics service processes events
Storage: Time-series database (InfluxDB)
```

10. Security Considerations

- **Rate Limiting:** 100 URLs per user per hour
- **Spam Prevention:** Check against blacklisted domains
- **Analytics Privacy:** Hash IP addresses
- **Custom URLs:** Validate against reserved words

11. Monitoring

Key Metrics:

- URL creation rate
- Redirection latency

- Cache hit rate
- Database query performance
- Error rates

This design can handle 100M URLs daily with proper caching and database optimization."

---

13. Design a Chat Application like WhatsApp

Answer Format:

```
Requirements → Architecture → Database → Real-time → Scaling
```

Sample Answer: "I'll design a chat application like WhatsApp focusing on real-time messaging.

1. Requirements Analysis

Functional Requirements:

- Send/receive messages instantly
- One-on-one and group chats
- Online/offline status
- Message history
- Push notifications
- File sharing (images, videos)

Non-Functional Requirements:

- **Users:** 2 billion users
- **Messages:** 100 billion messages daily
- **Latency:** <100ms for message delivery
- **Availability:** 99.99% uptime
- **Consistency:** Messages must be delivered in order

2. Capacity Estimation

```
Daily Active Users: 2B
Messages per user per day: 50
Total daily messages: 100B
Messages per second: 1.16M
Peak load: 3.5M messages/second
Message size: 100 bytes average
Daily storage: 100B × 100 bytes = 10TB
```

3. High-Level Architecture

```
[Mobile App] → [Load Balancer] → [API Gateway] → [Chat Service]
                                → [User Service]
                                → [Notification Service]

[Chat Service] → [Message Queue] → [Database]
          → [Redis Cache]
          → [File Storage]
```

4. Database Design

User Table:

sql

```sql
CREATE TABLE users (
    user_id BIGINT PRIMARY KEY,
    phone_number VARCHAR(15) UNIQUE,
    username VARCHAR(50),
    last_seen TIMESTAMP,
    is_online BOOLEAN,
    created_at TIMESTAMP
);
```

Messages Table (NoSQL - Cassandra):

```
{
  message_id: UUID,
  chat_id: UUID,
  sender_id: BIGINT,
  content: TEXT,
  message_type: ENUM('text', 'image', 'video', 'audio'),
  timestamp: TIMESTAMP,
  status: ENUM('sent', 'delivered', 'read')
}

Partition Key: chat_id
Clustering Key: timestamp
```

Chats Table:

```sql
CREATE TABLE chats (
  chat_id UUID PRIMARY KEY,
  chat_type ENUM('private', 'group'),
  created_at TIMESTAMP,
  updated_at TIMESTAMP
);
```

Chat Participants:

```sql
CREATE TABLE chat_participants (
  chat_id UUID,
  user_id BIGINT,
  joined_at TIMESTAMP,
  role ENUM('admin', 'member'),
  PRIMARY KEY (chat_id, user_id)
);
```

5. Real-time Communication

WebSocket Connection Management:

```python
```

```python
class ConnectionManager:
    def __init__(self):
        self.connections = {}  # user_id -> websocket

    async def connect(self, user_id, websocket):
        self.connections[user_id] = websocket
        await self.update_user_status(user_id, 'online')

    async def disconnect(self, user_id):
        if user_id in self.connections:
            del self.connections[user_id]
            await self.update_user_status(user_id, 'offline')

    async def send_message(self, user_id, message):
        if user_id in self.connections:
            await self.connections[user_id].send_text(message)
```

Message Flow:

1. User A sends message
2. Message saved to database
3. Message queued for delivery
4. Check if User B is online
5. If online: Send via WebSocket
6. If offline: Send push notification
7. Update message status (delivered/read)

6. Message Delivery System

Message Queue (Apache Kafka):

python

```python
# Producer
async def send_message(chat_id, sender_id, content):
    message = {
        'message_id': generate_uuid(),
        'chat_id': chat_id,
        'sender_id': sender_id,
        'content': content,
        'timestamp': datetime.now(),
        'status': 'sent'
    }

    # Save to database
    await database.save_message(message)

    # Queue for delivery
    await kafka_producer.send('message_delivery', message)
```

Consumer:

```python
async def process_message_delivery(message):
    # Get chat participants
    participants = await get_chat_participants(message['chat_id'])

    for participant in participants:
        if participant != message['sender_id']:
            await deliver_message(participant, message)
```

7. Caching Strategy

Redis Cache Layers:

```python
```

```
# Recent messages cache
key: f"chat:{chat_id}:recent"
value: List of last 50 messages
ttl: 24 hours

# User online status
key: f"user:{user_id}:status"
value: {'online': True, 'last_seen': timestamp}
ttl: 30 minutes

# Chat metadata
key: f"chat:{chat_id}:meta"
value: {participants, chat_type, settings}
ttl: 1 hour
```

8. Scaling Solutions

Database Sharding:

```python
# Shard messages by chat_id
def get_shard(chat_id):
    return hash(chat_id) % num_shards

# Consistent hashing for user data
def get_user_shard(user_id):
    return consistent_hash(user_id)
```

Service Partitioning:

```
Chat Service: Handle message sending/receiving
User Service: Handle user management, status
Notification Service: Handle push notifications
File Service: Handle media uploads
Analytics Service: Handle message analytics
```

9. Push Notifications

Notification Flow:

```python
```

```python
async def send_notification(user_id, message):
    # Check if user is online
    if not await is_user_online(user_id):
        # Get device tokens
        devices = await get_user_devices(user_id)

        for device in devices:
            await push_service.send_notification(
                token=device.token,
                title=message['sender_name'],
                body=message['content'][:100],
                data={'chat_id': message['chat_id']}
            )
```

10. File Sharing

File Upload Process:

python

```python
async def upload_file(file, chat_id):
    # Generate unique filename
    filename = f"{chat_id}/{uuid4()}.{file.extension}"

    # Upload to object storage (AWS S3)
    file_url = await s3_client.upload(file, filename)

    # Create message with file reference
    message = {
        'chat_id': chat_id,
        'message_type': 'file',
        'content': file_url,
        'metadata': {
            'filename': file.name,
            'size': file.size,
            'mime_type': file.mime_type
        }
    }

    return await send_message(message)
```

11. Message Status Tracking

Status Updates:

python

```
class MessageStatus:
    SENT = 'sent'
    DELIVERED = 'delivered'
    READ = 'read'

async def update_message_status(message_id, status):
    await database.update_message_status(message_id, status)
    await notify_sender(message_id, status)
```

12. Security Considerations

- **End-to-end encryption** for message content
- **Rate limiting** to prevent spam
- **Input validation** for all message content
- **Authentication** via phone number verification
- **Message expiration** for sensitive chats

13. Monitoring & Analytics

Key Metrics:

- Message delivery rate
- Average delivery time
- Connection uptime
- Database query performance
- WebSocket connection health

This design handles 2 billion users with proper sharding, caching, and real-time communication infrastructure."

---

14. Design a Social Media Feed like Instagram

Answer Format:

```
Requirements → Architecture → Feed Generation → Storage → Scaling
```

Sample Answer: "I'll design a social media feed system like Instagram focusing on timeline generation.

1. Requirements Analysis

Functional Requirements:

- Users can post photos/videos

- Users can follow other users
- Generate personalized timeline/feed
- Like, comment, share posts
- Real-time feed updates
- Trending/explore page

Non-Functional Requirements:

- **Users:** 1 billion users
- **Posts:** 500 million posts daily
- **Timeline generation:** <200ms
- **Availability:** 99.9% uptime
- **Storage:** Photos/videos for 10 years

2. Capacity Estimation

```
Daily Active Users: 500M
Posts per user per day: 1 (on average)
Total daily posts: 500M
Timeline requests per user: 20
Daily timeline requests: 10B
Average post size: 1MB
Daily storage: 500GB
```

3. High-Level Architecture

```
[Mobile App] → [CDN] → [Load Balancer] → [API Gateway]
                                       → [User Service]
                                       → [Post Service]
                                       → [Timeline Service]
                                       → [Notification Service]
```

4. Database Design

Users Table:

```sql
```

```
CREATE TABLE users (
    user_id BIGINT PRIMARY KEY,
    username VARCHAR(50) UNIQUE,
    email VARCHAR(100),
    profile_pic_url TEXT,
    follower_count INT DEFAULT 0,
    following_count INT DEFAULT 0,
    created_at TIMESTAMP
);
```

Posts Table (NoSQL - Cassandra):

```
{
    post_id: UUID,
    user_id: BIGINT,
    content_url: TEXT,
    caption: TEXT,
    post_type: ENUM('photo', 'video', 'story'),
    timestamp: TIMESTAMP,
    like_count: INT,
    comment_count: INT,
    location: TEXT
}

Partition Key: user_id
Clustering Key: timestamp (descending)
```

Follows Table:

sql

```
CREATE TABLE follows (
    follower_id BIGINT,
    following_id BIGINT,
    created_at TIMESTAMP,
    PRIMARY KEY (follower_id, following_id)
);

CREATE INDEX idx_following ON follows(following_id);
```

5. Feed Generation Strategies

Strategy 1: Pull Model (Read-time)

- Generate feed when user requests it

- Query posts from all people user follows
- Sort by timestamp and relevance

```python
async def generate_feed_pull(user_id, limit=20):
    # Get users that current user follows
    following = await get_following_list(user_id)

    # Get recent posts from all following users
    posts = []
    for followed_user in following:
        user_posts = await get_user_posts(followed_user, limit=10)
        posts.extend(user_posts)

    # Sort by timestamp and relevance
    sorted_posts = sort_by_relevance(posts)
    return sorted_posts[:limit]
```

Strategy 2: Push Model (Write-time)

- Pre-generate feed when posts are created
- Store personalized feeds for each user

```python
async def create_post_push(user_id, post_data):
    # Create post
    post = await save_post(user_id, post_data)

    # Get followers
    followers = await get_followers(user_id)

    # Add post to each follower's feed
    for follower in followers:
        await add_to_feed(follower, post)
```

Strategy 3: Hybrid Model (Instagram's Approach)

- **Push for regular users:** Pre-generate feeds
- **Pull for celebrities:** Generate on-demand (too many followers)
- **Cache popular content:** Store trending posts separately

6. Timeline Storage (Redis)

```python
# User timeline storage
key: f"timeline:{user_id}"
value: [
    {post_id, user_id, timestamp, score},
    {post_id, user_id, timestamp, score},
    ...
]
structure: Sorted Set (by score/timestamp)
size: Keep latest 1000 posts per user
```

7. Content Storage

Media Files:

```
Original Photos/Videos: AWS S3
Thumbnails: CDN (CloudFront)
Metadata: Database

Storage Strategy:
- Multiple resolutions (thumbnail, medium, full)
- Geographic distribution via CDN
- Compression for mobile users
```

8. Ranking Algorithm

```python
```

```
def calculate_post_score(post, user_preferences):
    score = 0

    # Recency (newer posts ranked higher)
    time_decay = calculate_time_decay(post.timestamp)
    score += time_decay * 0.3

    # Engagement (likes, comments, shares)
    engagement_score = (post.likes + post.comments * 2 + post.shares * 3)
    score += engagement_score * 0.4

    # User affinity (how much user interacts with poster)
    affinity = get_user_affinity(user_id, post.user_id)
    score += affinity * 0.2

    # Content type preference
    content_pref = get_content_preference(user_id, post.type)
    score += content_pref * 0.1

    return score
```

9. Scaling Solutions

Database Sharding:

```
python

# Posts sharded by user_id
def get_post_shard(user_id):
    return user_id % 1000

# Timelines sharded by user_id
def get_timeline_shard(user_id):
    return user_id % 100
```

Caching Layers:

```
L1: Application Cache (User timeline)
L2: Redis Cluster (Recent posts, user data)
L3: CDN (Media files, static content)
```

10. Real-time Updates

```
python
```

```
# WebSocket for real-time feed updates
async def notify_new_post(user_id, post):
    followers = await get_active_followers(user_id)

    for follower in followers:
        if is_user_online(follower):
            await websocket_send(follower, {
                'type': 'new_post',
                'post': post
            })
```

This design handles 1 billion users with proper feed generation, caching, and real-time features."

---

15. How would you design a distributed cache system like Redis Cluster?

Answer Format:

```
Requirements → Architecture → Consistency → Partitioning → Fault Tolerance
```

Sample Answer: "I'll design a distributed cache system focusing on high availability and performance.

1. Requirements

Functional Requirements:

- Store key-value pairs in memory
- GET/SET operations
- TTL (Time To Live) support
- Data partitioning across nodes
- Automatic failover

Non-Functional Requirements:

- **Latency:** <1ms for cache operations
- **Throughput:** 1M operations per second
- **Availability:** 99.99% uptime
- **Consistency:** Eventually consistent
- **Memory:** Support TBs of data

2. High-Level Architecture

```
[Client] → [Client Library] → [Node 1] → [Memory]
                            → [Node 2] → [Memory]
                            → [Node 3] → [Memory]

[Configuration Service] → [All Nodes]
[Monitoring Service] → [All Nodes]
```

3. Data Partitioning (Consistent Hashing)

python

```python
class ConsistentHash:
    def __init__(self, nodes, replicas=3):
        self.replicas = replicas
        self.ring = {}
        self.sorted_keys = []

        for node in nodes:
            self.add_node(node)

    def add_node(self, node):
        for i in range(self.replicas):
            key = self.hash(f"{node}:{i}")
            self.ring[key] = node
            self.sorted_keys.append(key)
        self.sorted_keys.sort()

    def get_node(self, key):
        if not self.ring:
            return None

        hash_key = self.hash(key)

        # Find first node clockwise
        for ring_key in self.sorted_keys:
            if hash_key <= ring_key:
                return self.ring[ring_key]

        # Wrap around to first node
        return self.ring[self.sorted_keys[0]]
```

4. Replication Strategy

python

```python
class ReplicationManager:
    def __init__(self, hash_ring, replication_factor=3):
        self.hash_ring = hash_ring
        self.replication_factor = replication_factor

    def get_replica_nodes(self, key):
        primary_node = self.hash_ring.get_node(key)
        replicas = [primary_node]

        # Get next N-1 nodes in ring
        current_pos = self.hash_ring.sorted_keys.index(
            self.hash_ring.hash(key)
        )

        for i in range(1, self.replication_factor):
            next_pos = (current_pos + i) % len(self.hash_ring.sorted_keys)
            next_key = self.hash_ring.sorted_keys[next_pos]
            replicas.append(self.hash_ring.ring[next_key])

        return replicas
```

5. Cache Operations

python

```python
class CacheNode:
    def __init__(self, node_id):
        self.node_id = node_id
        self.data = {}  # In-memory storage
        self.ttl_data = {}  # TTL tracking

    async def set(self, key, value, ttl=None):
        self.data[key] = value

        if ttl:
            expire_time = time.time() + ttl
            self.ttl_data[key] = expire_time

        # Replicate to other nodes
        await self.replicate_write(key, value, ttl)

    async def get(self, key):
        # Check if expired
        if key in self.ttl_data:
            if time.time() > self.ttl_data[key]:
                await self.delete(key)
                return None

        return self.data.get(key)

    async def delete(self, key):
        self.data.pop(key, None)
        self.ttl_data.pop(key, None)
        await self.replicate_delete(key)
```

6. Fault Tolerance

Health Checking:

```python
```

```python
class HealthChecker:
    async def check_node_health(self, node):
        try:
            # Send ping request
            response = await node.ping(timeout=1)
            return response == "PONG"
        except TimeoutError:
            return False

    async def monitor_cluster(self):
        while True:
            for node in self.nodes:
                is_healthy = await self.check_node_health(node)

                if not is_healthy:
                    await self.handle_node_failure(node)

            await asyncio.sleep(5)  # Check every 5 seconds
```

Failover Process:

```python
async def handle_node_failure(self, failed_node):
    # 1. Mark node as failed
    self.mark_node_failed(failed_node)

    # 2. Redistribute data to surviving replicas
    for key in failed_node.get_keys():
        replica_nodes = self.get_replica_nodes(key)
        surviving_replicas = [n for n in replica_nodes if n != failed_node]

        if surviving_replicas:
            # Promote replica to primary
            await self.promote_replica(key, surviving_replicas[0])

    # 3. Update hash ring
    self.hash_ring.remove_node(failed_node)

    # 4. Notify clients about topology change
    await self.notify_clients_topology_change()
```

7. Client Library

```python
```

```
class CacheClient:
    def __init__(self, cluster_nodes):
        self.hash_ring = ConsistentHash(cluster_nodes)
        self.connections = {}

    async def set(self, key, value, ttl=None):
        node = self.hash_ring.get_node(key)
        connection = await self.get_connection(node)

        try:
            await connection.set(key, value, ttl)
        except ConnectionError:
            # Try next replica
            replicas = self.get_replica_nodes(key)[1:]
            for replica in replicas:
                try:
                    conn = await self.get_connection(replica)
                    await conn.set(key, value, ttl)
                    break
                except ConnectionError:
                    continue

    async def get(self, key):
        replicas = self.get_replica_nodes(key)

        for node in replicas:
            try:
                connection = await self.get_connection(node)
                value = await connection.get(key)

                if value is not None:
                    return value
            except ConnectionError:
                continue

        return None
```

8. Configuration Management

python

```python
class ConfigManager:
    def __init__(self):
        self.cluster_config = {
            'nodes': [],
            'replication_factor': 3,
            'hash_algorithm': 'md5',
            'failure_detection_interval': 5
        }

    async def add_node(self, node):
        self.cluster_config['nodes'].append(node)
        await self.redistribute_data(node)
        await self.notify_cluster_change()

    async def remove_node(self, node):
        self.cluster_config['nodes'].remove(node)
        await self.redistribute_data()
        await self.notify_cluster_change()
```

9. Memory Management

```python
class MemoryManager:
    def __init__(self, max_memory_mb):
        self.max_memory = max_memory_mb * 1024 * 1024  # Convert to bytes
        self.current_memory = 0
        self.lru_tracker = OrderedDict()

    def evict_if_needed(self, new_size):
        while (self.current_memory + new_size) > self.max_memory:
            # LRU eviction
            oldest_key = next(iter(self.lru_tracker))
            self.evict_key(oldest_key)

    def evict_key(self, key):
        size = self.get_key_size(key)
        del self.data[key]
        del self.lru_tracker[key]
        self.current_memory -= size
```

This design provides a scalable, fault-tolerant distributed cache with automatic partitioning and replication."

System Design Case Studies

Case Study 1: Scaling from 1K to 1M Users

Scenario: You have a social media app that started with 1,000 users and now needs to scale to 1 million users.

Answer Approach: "I'll walk through the scaling journey step by step, showing what changes at each stage.

Stage 1: 1K Users - Single Server

```
Architecture: [Users] → [Single Server] → [Database on same server]
Problems: None yet
Cost: $50/month
```

Stage 2: 10K Users - Separate Database

```
Architecture: [Users] → [App Server] → [Database Server]
Problems: Database becomes bottleneck
Changes: Move database to separate server
Cost: $200/month
```

Stage 3: 100K Users - Add Load Balancer

```
Architecture: [Users] → [Load Balancer] → [App Server 1]
                      → [App Server 2]
                      → [Database Server]
Problems: Database read overload
Changes: Multiple app servers + load balancer
Cost: $800/month
```

Stage 4: 500K Users - Add Caching

```
Architecture: [Users] → [Load Balancer] → [App Servers] → [Redis Cache]
                                        → [Master DB]
                                        → [Read Replicas]
Problems: Write operations slow
Changes: Redis cache + read replicas
Cost: $2,500/month
```

Stage 5: 1M Users - Microservices

```
Architecture: [Users] → [API Gateway] → [User Service]
                                      → [Post Service]
                                      → [Feed Service]
                                      → [Notification Service]
Problems: Different services need different scaling
Changes: Break into microservices, separate databases
Cost: $8,000/month
```

Key Lessons:

- Scale based on actual problems, not predictions
- Monitor metrics to identify bottlenecks
- Each stage solves specific problems
- Cost increases with complexity"

How to Answer System Design Questions

Step-by-Step Framework

Step 1: Clarify Requirements (5 minutes)

```
Ask questions like:
- How many users?
- What features are most important?
- What's the read/write ratio?
- Any specific latency requirements?
- Budget constraints?
```

Step 2: Estimate Scale (5 minutes)

```
Calculate:
- Daily/Monthly active users
- Requests per second
- Data storage requirements
- Bandwidth needs

Example:
"100M users × 10 requests/day = 1B requests/day = 11,500 RPS"
```

Step 3: High-Level Design (10 minutes)

Draw basic architecture:
[Client] → [Load Balancer] → [Servers] → [Database]

Include:
- Major components
- Data flow
- Basic connections

Step 4: Database Design (10 minutes)

Design key tables:
- Choose SQL vs NoSQL
- Define main entities
- Show relationships
- Consider indexing

Step 5: Deep Dive (15 minutes)

Focus on 2-3 components:
- Detailed API design
- Caching strategy
- Database sharding
- Message queues

Step 6: Scale the Design (10 minutes)

Address bottlenecks:
- Add load balancers
- Database replication
- Caching layers
- CDN for static content

Step 7: Address Edge Cases (5 minutes)

Consider:
- What happens if a server fails?
- How to handle traffic spikes?
- Data consistency issues
- Security concerns

Common Mistakes to Avoid

❌ **Don't:**

- Start coding immediately
- Design for billions of users from day 1
- Ignore the requirements
- Get lost in minor details
- Forget about failures

✅ **Do:**

- Ask clarifying questions
- Start simple, then scale
- Explain your thought process
- Consider trade-offs
- Think about monitoring

Sample Answer Structure

"Let me design a [system name] step by step.

First, let me clarify the requirements...
[Ask 3-4 questions]

Based on your answers, I'll estimate the scale...
[Show calculations]

Here's my high-level design...
[Draw architecture]

For the database, I'll use...
[Explain choice and design]

To handle [specific challenge], I'll implement...
[Show detailed solution]

To scale this system, I would...
[Discuss scaling strategy]

Some edge cases to consider are...
[Address potential issues]

Would you like me to deep dive into any specific component?"

Interview Tips

Communication:

- Think out loud
- Ask questions when stuck
- Explain trade-offs
- Be honest about what you don't know

Time Management:

- Spend adequate time on requirements
- Don't get stuck on one component
- Save time for scaling discussion
- Practice with a timer

Technical Depth:

- Know your basics well
- Understand when to use each technology
- Be able to estimate numbers quickly
- Think about real-world constraints

Practice Questions:

1. Design Twitter
2. Design Uber
3. Design Netflix
4. Design a messaging system
5. Design a payment system
6. Design a search engine
7. Design a recommendation system
8. Design a video streaming service

Remember: There's no single "correct" answer. Focus on demonstrating your thought process, technical knowledge, and ability to build scalable systems!